

Express

Overview

- Minimal node js framework
- Features
 - complex routing
 - req/resp handling
 - middleware
 - server side rendering
- Rapid app development
 - MVC Architecture

basic app

```

1  const express = require('express');
2
3  const app = express();
4
5  app.get('/', (req, res) => {
6    console.log(`Request received`);
7    res.status(200).send('Hello from the server');
8  });
9
10 const port = 3000;
11 app.listen(port, () => {
12   console.log(`App running on port ${port}`);
13 });

```

send file

```
const app = express();

app.get('/', (req, res) => {
  res.status(200).sendFile(path.join(__dirname, '/index.html'));
});

const port = 3000;

app.listen(port, () => {
  console.log(`App running on port ${port}`);
});
```

parameters

Static Files

```
app.use(express.static('public'));
```

Per gestire i file statici, quali immagini, file CSS e file JavaScript, utilizzare la funzione middleware integrata `express.static` in Express.

Fornire il nome della directory che contiene gli asset statici alla funzione middleware `express.static` per iniziare a gestire i file direttamente. Ad esempio, utilizzare il seguente codice per gestire le immagini, i file CSS e i file JavaScript nella directory denominata `public`:

```
app.use(express.static('public'));
app.use(express.static('images'));
```

```
app.use('/static', express.static('public'));
```

json response

```
app.get('/', (req, res) => {  
  console.log(`Request received`);  
  res.status(200).json({ message: 'Hello from the server' });  
});
```

Simple post

```
app.post('/', (req, res) => {
  console.log(`Request received`);
  res.status(404).json({ message: 'Post Endpont!!!' });
});
```

Architettura REST

- **REST** (*RE*presentational State *T*ransfer)
- insieme di linee guida o principi per la realizzazione di una architettura di sistema
 - uno stile architetturale
- non si riferisce ad un sistema concreto
- non si tratta di uno standard

Principi REST

- Identificazione delle risorse
- Utilizzo esplicito dei metodi HTTP
- Risorse autodescrittive
- Collegamenti tra risorse
- Comunicazione senza stato

Esempio risorse

products

http://my-url/addNewProduct

/getProduct

/updateProduct

/deleteProduct

Esempio risorse

products

users

orders

`http://my-url/addNewProduct`

`/getProduct`

`/updateProduct`

`/deleteProduct`

`/getProductbyOrder`

`/getOrderByUser`

CRUD Operations

/addNewProduct

/getProduct

/updateProduct

/deleteProduct

/getProductbyOrder

/getOrderbyUser

POST /products

GET /products/3

PUT /products/3

PATCH /products/3

DELETE /products/3

GET /orders/4/products

GET /users/9/orders

JSON formatting

```
{  
  "id": 1,  
  "name": "cerulean",  
  "year": 2000,  
  "color": "#98B2D1",  
  "pantone_value": "15-4020"  
}
```

JSON formatting

JSEND

<https://github.com/omniti-labs/jsend>

```
{
  "id": 1,
  "name": "cerulean",
  "year": 2000,
  "color": "#98B2D1",
  "pantone_value": "15-4020"
}
```



```
{
  "status": "success",
  "data": {
    "id": 1,
    "name": "cerulean",
    "year": 2000,
    "color": "#98B2D1",
    "pantone_value": "15-4020"
  }
}
```

JSON formatting

JSEND

<https://github.com/omniti-labs/jsend>

```
{
  "id": 1,
  "name": "cerulean",
  "year": 2000,
  "color": "#98B2D1",
  "pantone_value": "15-4020"
}
```



```
{
  "status": "success",
  "data": {
    "id": 1,
    "name": "cerulean",
    "year": 2000,
    "color": "#98B2D1",
    "pantone_value": "15-4020"
  }
}
```

1. [JSON API](#) - JSON API covers creating and updating resources as well, not just responses.
2. [JSend](#) - Simple and probably what you are already doing.
3. [OData JSON Protocol](#) - Very complicated.

Stateless!!!

- Lo stato va mantenuto nel client
 - Il server per rispondere non deve ricordare una richiesta precedente
- Esempio
 - paging:
 - <https://reqres.in/api/users?page=1>
 - ~~<https://reqres.in/api/users?page=nextpage>~~
 - login
 - ogni richiesta è autenticata singolarmente

CRUD API

GET

```
app.get('/api/v1/products', (req, res) => {  
  res.status(200).json({  
    status: 'success',  
    data: {  
      products: products,  
    },  
  });  
});
```

GET

```
app.get('/api/v1/products/:id', (req, res) => {
  console.log(req.params);

  const prod = products.find((el) => el.id == req.params.id);
  console.log(prod);
  if (prod == undefined) {
    res.status(404).json({
      status: 'fail',
      message: 'ID non trovato',
    });
  } else {
    res.status(200).json({
      status: 'success',
      data: {
        product: prod,
      },
    });
  }
});
```

POST

```
app.post('/api/v1/products', (req, res) => {
  const newId = products[products.length - 1].id + 1;
  const newProd = Object.assign({ id: newId }, req.body);

  products.push(newProd);
  res.status(201).json({
    status: 'success',
    data: { product: newProd },
  });
});
```

PUT/PATCH

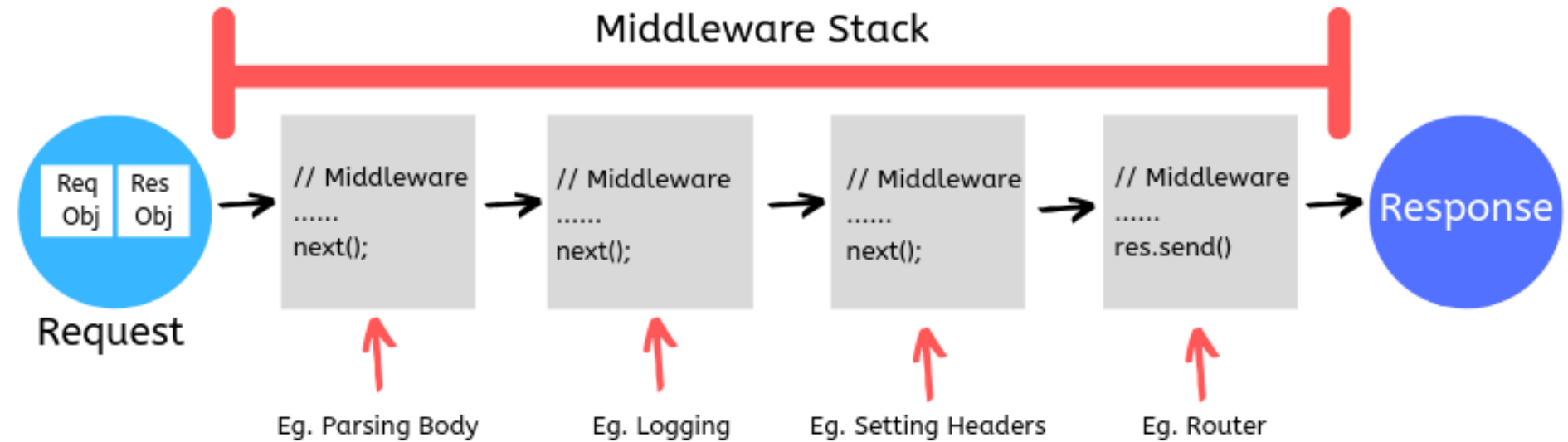
```
app.patch('/api/v1/products/:id', (req, res) => {
  const prod = products.find((el) => el.id == req.params.id);
  if (prod == undefined) {
    res.status(404).json({
      status: 'fail',
      message: 'ID non trovato',
    });
  } else {
    // Update ....

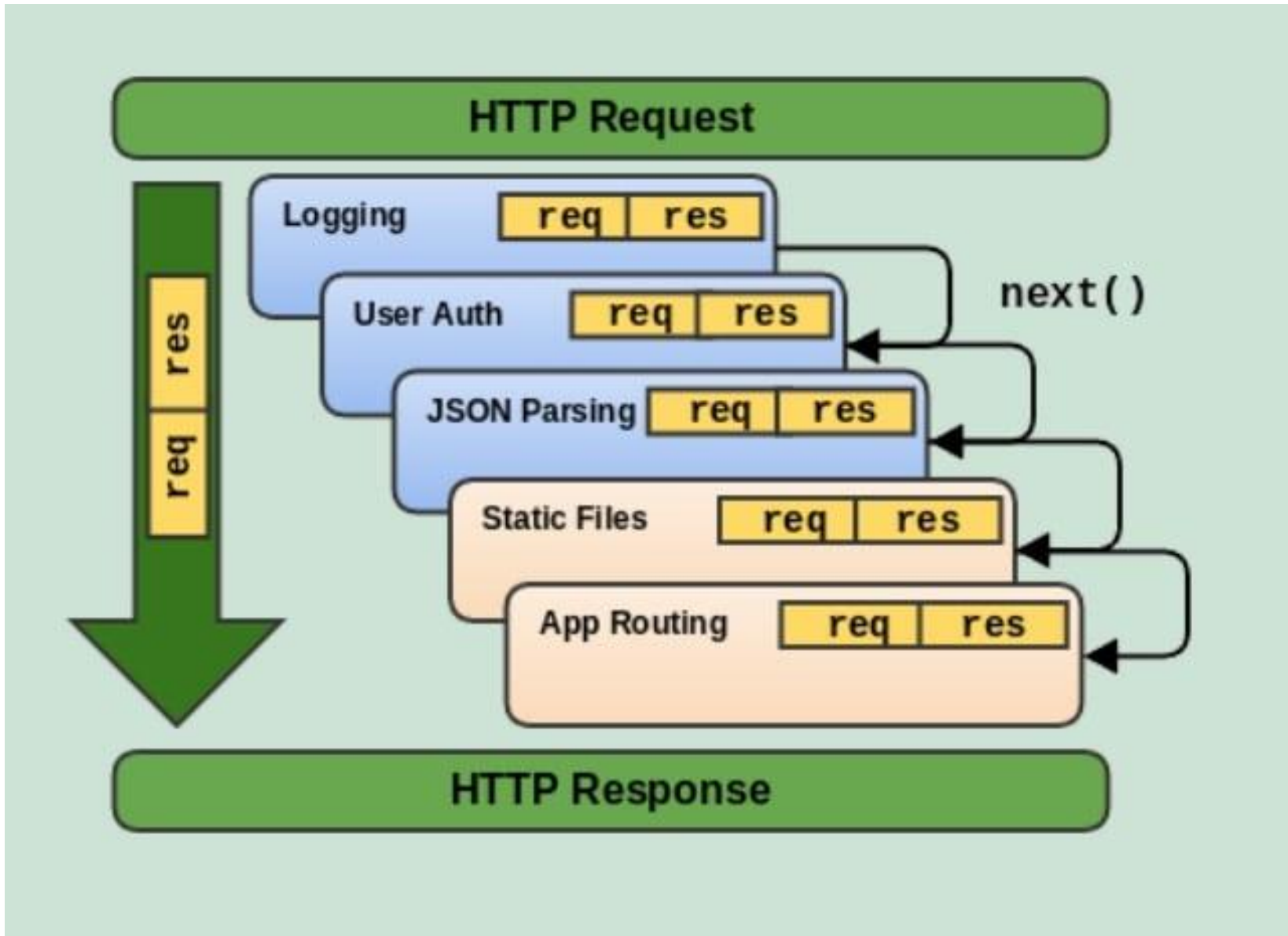
    res.status(200).json({
      status: 'success',
      data: {
        product: prod,
      },
    });
  }
});
```

DELETE

```
app.delete('/api/v1/products/:id', (req, res) => {
  const prod = products.find((el) => el.id == req.params.id);
  if (prod == undefined) {
    res.status(404).json({
      status: 'fail',
      message: 'ID non trovato',
    });
  } else {
    // Delete ....
    res.status(204).json({
      status: 'success',
      data: null,
    });
  }
});
```

Middleware





Custom Middleware

```
app.use(function (req, res, next) {
  console.log('Hello from the middleware !');
  next();
});

app.use('/api', function (req, res, next) {
  console.log('This middleware handles the data route');
  next();
});
```

Third Party middleware

```
const morgan = require('morgan');  
app.use(morgan('dev'));
```

npm install morgan

<https://github.com/expressjs/morgan>

Routing

<https://expressjs.com/en/guide/routing.html>

https://developer.mozilla.org/en-US/docs/Learn/Server-side/Express_Nodejs/routes

get(), post(), put()...

```
// GET method route
app.get('/', function (req, res) {
  res.send('GET request to the homepage');
});
```

```
// POST method route
app.post('/', function (req, res) {
  res.send('POST request to the homepage');
});
```

- get, post, put, head, delete, options, trace, copy, lock, mkcol, move, purge, propfind, proppatch, unlock, report, mkactivity, checkout, merge, msearch, notify, subscribe, unsubscribe, patch, search e connect

multiple handlers

```
var cb0 = function (req, res, next) {
  console.log('CB0');
  next();
}

var cb1 = function (req, res, next) {
  console.log('CB1');
  next();
}

var cb2 = function (req, res) {
  res.send('Hello from C!');
}

app.get('/example/c', [cb0, cb1, cb2]);
```

Un array di funzioni callback possono gestire una route.

Order of routes

```
app.get("/", (req, res) => {
  res.send("Home page");
});

app.get("/page", (req, res) => {
  res.send("A static page");
});

app.get("/:post", (req, res) => {
  res.send("Single post");
});

app.get("*", (req, res) => {
  res.send("Any");
});
```



```
app.get("/", (req, res) => {
  res.send("Home page");
});

app.get("/:post", (req, res) => {
  res.send("Single post");
});

app.get("/page", (req, res) => {
  res.send("A static page");
});

app.get("*", (req, res) => {
  res.send("Any");
});
```

- parametric path inserted just before a literal one takes the precedence over the literal one

route()

- È possibile creare handler di route concatenabili per un percorso di route, utilizzando **app.route()**.

```
app.route('/book')
    .get(function(req, res) {
        res.send('Get a random book');
    })
    .post(function(req, res) {
        res.send('Add a book');
    })
    .put(function(req, res) {
        res.send('Update the book');
    });
```

Routers

```
var express = require('express');
var router = express.Router();

// middleware that is specific to this router
router.use(function timeLog(req, res, next) {
  console.log('Time: ', Date.now());
  next();
});

// define the home page route
router.get('/', function(req, res) {
  res.send('Birds home page');
});

// define the about route
router.get('/about', function(req, res) {
  res.send('About birds');
});

module.exports = router;
```

La classe `express.Router` crea handler di route modulari e montabili.

Un'istanza Router è un middleware e un sistema di routing completo; per questa ragione, spesso si definisce "mini-app".

`/birds/`

`/birds/about`

```
var birds = require('./birds');
...
app.use('/birds', birds);
```

param()

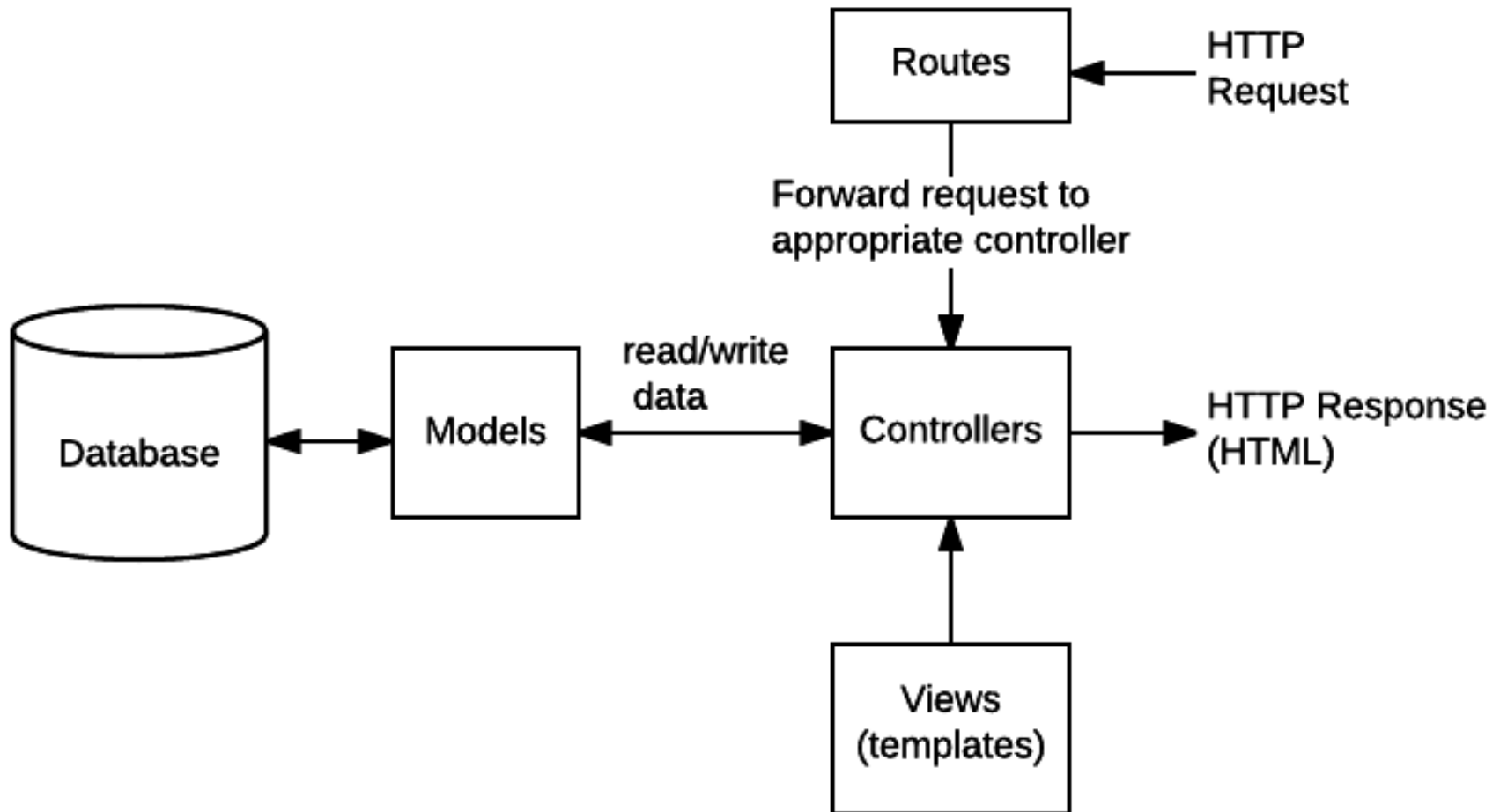
```
const express = require("express");
const router = express.Router();

router.param("userId", (req, res, next, id) => {
  console.log("This function will be called first");
  next();
});

router.get("/user/:userId", (req, res) => {
  console.log("Then this function will be called");
  res.end();
});

// Export router
module.exports = router;
```

MVC Architecture



Environment

- Environment variables allowing apps to behave differently based on the environment
- Externalize all environment specific parameters
- Examples
 - Which HTTP port to listen on
 - What path and folder your files are located in, that you want to serve
 - Pointing to a development, staging, test, or production database

Examples

```
const port = process.env.PORT;

app.listen(port, () => {
  console.log(`App running on port ${port}`);
});
```

- PORT=8626 node server.js
- PORT=8626 NODE_ENV=development node server.js

```
console.log(app.get('env'));

if (app.get('env') == 'development') {
  app.use(morgan('dev'));
}
```

.env file

.env

```
PORT=8765  
NODE_ENV=development  
  
USERNAME=loreti  
PASSWORD=12345
```

server.js

```
const dotenv = require('dotenv');  
dotenv.config();
```

```
const dotenv = require('dotenv');  
dotenv.config({ path: '/custom/path/to/.env' });
```

config module

config.js

```
const dotenv = require('dotenv');
dotenv.config();

module.exports = {
  username: process.env.USER_NAME,
  password: process.env.PASSWORD,
  port: process.env.PORT,
};
```

server.js

```
const { port, username, password } = require('./config');
```